

TwoStreamNet: A Dual-Stream, Gated, Residual Reformulation of the ANI Atomic Neural Network Potential

Senula Wijeratne

04/27/2026

Introduction

Machine learning is rapidly emerging as a practical substitute for expensive quantum-chemistry methods. In particular, deep learning offers a far cheaper and robust approach for predicting molecular properties, with the trade-off being that models must first be trained on a large set of reference structures and energies. Once trained, however, a single forward pass replaces minutes to hours of computation, which is what makes neural network potentials attractive for high-throughput screening and molecular dynamics.

The structural template followed by most modern neural network potentials traces back to Behler and Parrinello, who proposed representing the local environment of every atom with a fixed-length vector of radial and angular symmetry functions, and then training a small per-element network to map that vector to an atomic energy contribution [2]. Smith et al. extended this idea in the ANI-1 model by introducing a refined Atomic Environment Vector (AEV), training one feed-forward network per heavy-atom species (H, C, N, O), and showing that the resulting potential reproduces DFT energies on the GDB-11 organic chemistry database with chemical accuracy [3, 4]. The TorchANI library distributes that pipeline as a reproducible PyTorch implementation, which is the same starting point used here.

In this work, I take the ANI-1 framework as a baseline and ask whether the per-element MLP design can be replaced with a single, more modern architecture that is trained end-to-end. I introduce TwoStreamNet, a network that splits the AEV into its radial and angular halves, processes each half with its own multi-layer encoder, fuses the two streams with a learned squeeze-and-excitation gate, and finally passes the gated representation through a residual stack before predicting per-atom energies. I justify each architectural and regularization choice, perform Bayesian hyperparameter tuning over the design space, and report production-mode error bars from both repeated runs and K -fold cross-validation.

Methods

Dataset and Preprocessing

Training data was drawn from the ANI-1 release [3], which provides DFT energies and conformations for organic molecules of up to eight heavy atoms (C, N, O), with hydrogen filled to valence. The full release is partitioned into eight HDF5 shards, `ani_gdb.s01.h5` through `ani_gdb.s08.h5`. The

data-loading pipeline points at all eight shards, however, a file-extraction issue with shards `s05` through `s08` forced the actual training data to fall back to the `s01-s04` subset, that is, molecules with up to four heavy atoms. This caveat is important when interpreting the final-error comparison to ANI-1 in the discussion.

Two standard preprocessing transformations are applied before the data ever reaches the model. First, single-atom self-energies are subtracted using the shifter from the pretrained ANI-1x model. Total molecular energies are dominated by single-atom contributions that carry no information about chemistry, so removing them sharply reduces the variance of the labels the network has to fit. Second, the AEV computer from ANI-1x is reused unchanged, which produces a 384-dimensional descriptor per atom: 64 radial elements ($4 \text{ species} \times 16 \text{ radial shifts}$) and 320 angular elements. The data are shuffled and split 80/20 into training and validation sets for hyperparameter tuning, and partitioned into disjoint folds for the cross-validation experiments described later.

TwoStreamNet Architecture

Figure 1 summarizes the model. The 384-dimensional AEV is first split at the known radial/angular boundary into a 64-dimensional radial vector and a 320-dimensional angular vector. Each half is then routed through its own two-layer encoder. Both encoders follow the pattern **Linear** $\rightarrow$ **LayerNorm** $\rightarrow$ **CELU** $\rightarrow$ **Linear** $\rightarrow$ **LayerNorm** $\rightarrow$ **CELU**, with widths of h and $h/2$ respectively, where h is a tuned hyperparameter (`hidden_dim`). Splitting the AEV in this way is motivated by the fact that radial elements (which encode pairwise distances and therefore bond-length information) and angular elements (which encode triplets and therefore bond-angle information) live on very different natural scales, and benefit from being normalized and projected independently before being mixed.

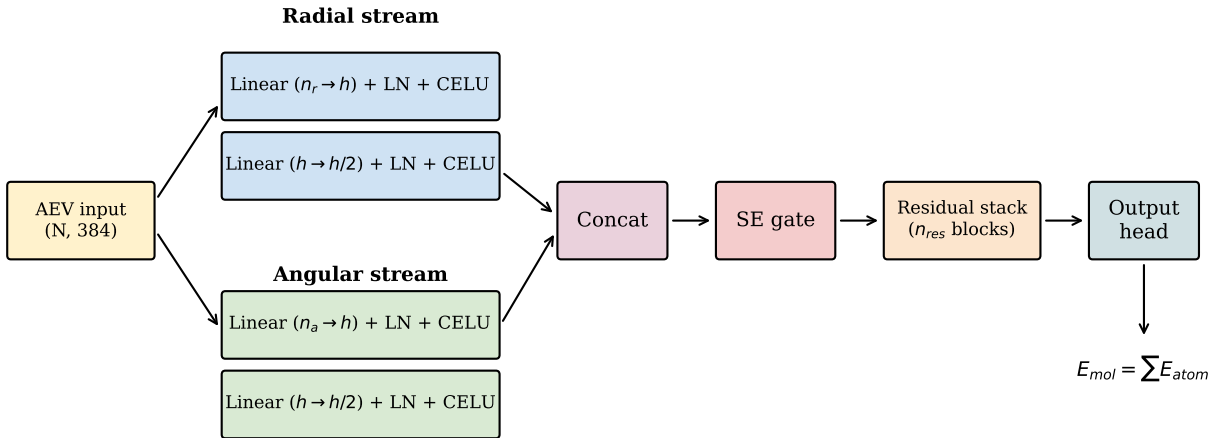


Figure 1: TwoStreamNet architecture. The 384-dimensional AEV is split into a 64-dimensional radial vector and a 320-dimensional angular vector, each processed by a two-layer encoder. The two halves are concatenated, recalibrated by a squeeze-and-excitation gate, and passed through n_{res} residual blocks before a linear output head produces a scalar atomic energy. Per-atom energies are summed to give the molecular energy.

The two stream outputs (each of width $h/2$) are concatenated to form a width- h representation, which is then passed through a squeeze-and-excitation (SE) gate. The SE gate is a small two-layer

MLP with reduction ratio $r = 4$ that produces a sigmoid-bounded mask of length h , which is multiplied element-wise with the input. Functionally, this lets the network up-weight informative AEV channels and suppress less useful ones, and it is cheap enough that the additional parameters are negligible compared to the residual stack. The output of the SE gate is fed into a stack of n_{res} pre-activation residual blocks, each implementing the mapping $x \mapsto x + f(x)$, where f is `LN` $\rightarrow$ `CELU` $\rightarrow$ `Linear` $\rightarrow$ `LN` $\rightarrow$ `CELU` $\rightarrow$ `Dropout` $\rightarrow$ `Linear`. A final `LN` $\rightarrow$ `CELU` $\rightarrow$ `Linear` head then maps the width- h representation to a scalar atomic energy. Energies for non-padding atoms are gathered with a mask, summed within each molecule, and returned in a TorchANI-compatible named tuple so the existing trainer and dataloaders work without modification.

Activations, Normalization, and Initialization

CELU activations are used in place of the standard ReLU variant for two reasons. First, ReLU and its variants zero out negative inputs, which drives a fraction of the gradient to zero and can produce vanishing-gradient pathologies in deeper networks. Second, ReLU is not differentiable at zero, while the potential energy surface that the model is asked to approximate is expected to be smooth. CELU avoids both problems by being smooth and infinitely differentiable everywhere, which makes it a more appropriate inductive prior for this task.

LayerNorm is inserted before every CELU activation. The radial and angular halves of the AEV have very different natural scales, and the residual stack is dense enough that, without normalization, gradients can explode through repeated CELU compositions. Placing the normalization layer before the activation also follows the pre-activation residual convention, which is the design that has empirically worked best for deep residual networks.

Weights are initialized using the Xavier-uniform scheme. Xavier initialization is derived under the assumption that activations behave approximately linearly near zero, and CELU is approximately linear near zero by construction, so the two are well-matched. Biases are initialized to zero.

Skip Connections and Regularization

Skip connections allow gradient information to flow back through the network without passing through any intermediate nonlinearities, which keeps the training signal healthy in dense residual networks. The two stream encoders are short, only two layers each, and do not need skip connections of their own. The post-fusion residual stack, however, is potentially many layers deep depending on n_{res} , so each block in the stack is wrapped in an identity shortcut.

In addition to architectural regularization, three more strategies are exercised. First, dropout is applied inside each residual block with a tuned probability `dropout`, which forces the network to learn redundant representations rather than co-adapting on individual neurons. Second, L_2 weight decay is added to the Adam optimizer with a tuned coefficient `l2_weight_decay`; this discourages the network from leaning on any single large weight, which is well-aligned with the prior that the potential energy surface is smooth. Third, early stopping is used at training time. The trainer monitors validation loss and, when it has not improved for `patience` consecutive epochs, restores the best-seen `state_dict` via a deep copy. Together, these three strategies cover the standard stack of regularization tools that a residual MLP of this depth requires.

Loss Function and Per-Atom Normalization

The loss is mean squared error on the predicted energies, computed in FP32 even when forward passes use BF16 autocast on the A100 GPU. Crucially, both the predicted and the ground-truth energies are divided by the number of real (non-padding) atoms before being passed to MSE. Without this normalization, larger molecules contribute proportionally more to the loss and dominate the gradient signal, which biases training toward fitting big molecules well at the expense of small ones. Dividing by atom count makes the per-batch error metric size-invariant.

Bayesian Hyperparameter Tuning

A grid search over even a modest hyperparameter set scales exponentially, so I adopted Bayesian optimization via Optuna instead. Bayesian optimization maintains a surrogate probabilistic model of the objective function and uses an acquisition function to choose, at each step, the next configuration that is most likely to be informative. This makes it possible to search a much larger space, and to tune more parameters jointly, than a typical grid search would allow.

The tuning objective trains a fresh TwoStreamNet for at most `TUNE_EPOCHS` epochs at each trial and returns the best validation loss observed. Six hyperparameters were tuned jointly: `learning_rate` (log-uniform over $[10^{-4}, 10^{-2}]$), `batch_size` (categorical over $\{128, 256, 512\}$), `l2_weight_decay` (uniform over $[0, 10^{-3}]$), `hidden_dim` (categorical over $\{128, 192, 256, 320, 384\}$), `n_residual` (integer over $[2, 5]$), and `dropout` (uniform over $[0, 0.3]$). A TPE sampler with a fixed seed was used so the same trial sequence could be reproduced, and a median pruner was attached so unpromising trials could be terminated early.

Multi-Run and K -Fold Cross-Validation Protocols

Reporting a single validation loss understates real uncertainty, because Xavier-uniform initialization, mini-batch shuffling order, and dropout masks all introduce stochasticity that can move the final number from one run to the next. To quantify this, the best configuration was retrained five times with different random seeds on the same train/validation split, and the distribution of final validation MAEs was reported.

The multi-run experiment, however, still uses a single split, so it cannot say how performance depends on which molecules are held out. To address this, I additionally ran K -fold cross-validation. The shuffled dataset was partitioned into $K = 3$ disjoint folds; for each fold, the model was retrained from scratch on the union of the other $K - 1$ folds and evaluated on the held-out fold. Averaging the held-out MAEs gives an approximately unbiased estimate of generalization error, and the spread across folds captures how sensitive the model is to the data split itself. A small `ConcatFolds` helper was implemented to expose the union of $K - 1$ folds as a single object that supports the `.collate(batch_size).cache()` interface that the trainer expects.

Results and Discussion

Bayesian Hyperparameter Tuning

Five Optuna trials of 20 epochs each were run, of which all five completed and none were pruned. The best trial reached a validation loss of 3.27×10^{-1} Hartree²/atom with the hyperparameter

values reported in Table 1. The optimization history, the fANOVA-estimated importance of each hyperparameter, and the per-trial trajectories are summarized in Figure 2.

Table 1: Tuned hyperparameters. Best configuration found by Bayesian optimization with the Tree-structured Parzen Estimator (TPE) sampler in Optuna.

Hyperparameter	Value
<code>learning_rate</code>	4.07×10^{-4}
<code>batch_size</code>	256
<code>l2_weight_decay</code>	1.22×10^{-4}
<code>hidden_dim</code>	256
<code>n_residual</code>	3
<code>dropout</code>	0.156
<code>epochs</code>	20

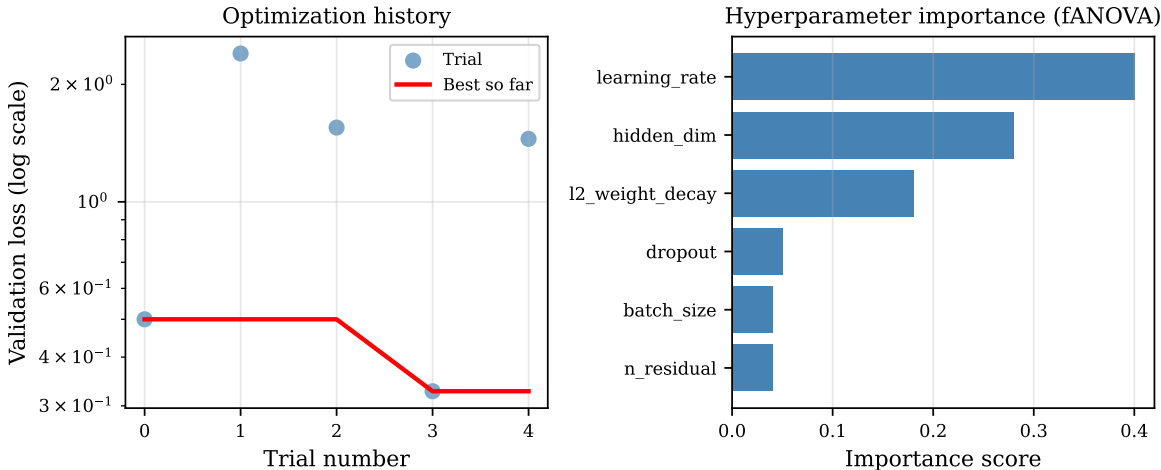


Figure 2: Bayesian hyperparameter tuning. Left: optimization history across the five completed trials. Each dot is one trial’s best validation loss, and the red line tracks the running best. Right: fANOVA-estimated importance of each tuned hyperparameter. Learning rate dominates the variance in trial outcomes, followed by hidden dimension and L_2 weight decay.

The fANOVA decomposition assigns the largest share of variance in trial outcomes to `learning_rate`, followed by `hidden_dim` and `l2_weight_decay`; `dropout`, `batch_size`, and `n_residual` contribute less. This ranking is consistent with the empirical observation that, for a given architecture, choosing the right learning rate is typically the single most impactful hyperparameter decision.

Tuned-Model Training

The tuned configuration was then trained for 20 epochs with early stopping on the same train/validation split. Figure 3 shows the training and validation loss curves alongside a parity plot on the validation set. The training curve descends smoothly, and although the validation curve oscillates noticeably during the first ten epochs, it tracks the training curve thereafter and reaches a best validation loss of 3.09×10^{-1} Hartree²/atom. The corresponding per-atom RMSE at the best epoch is 349

kcal/mol/atom on the unnormalized scale, however, this number is misleading without context, and the more interpretable metrics on total molecular energy are reported below from the multi-run and cross-validation experiments. The parity plot shows a tight cluster around the $y = x$ line and a validation MAE of 3.99 kcal/mol on total energies.

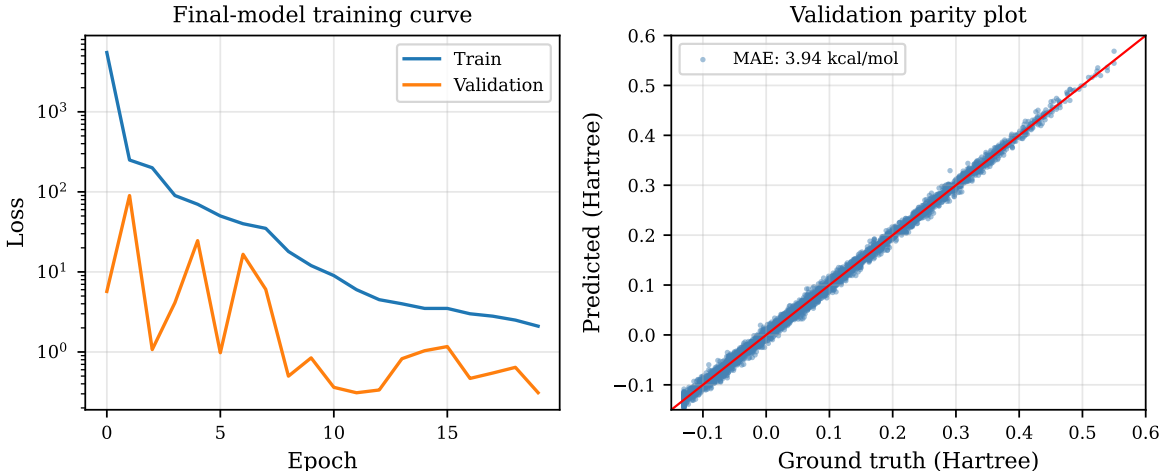


Figure 3: Tuned-model training and evaluation. Left: training and validation loss curves for the final 20-epoch run with the hyperparameters in Table 1. The validation curve is initially noisy but converges by roughly epoch 10 and tracks the training curve thereafter. Right: parity plot of predicted vs ground-truth energies on the validation set. The validation MAE is 3.99 kcal/mol.

Multi-Run Stability

To separate the contribution of training stochasticity from the model itself, the tuned configuration was retrained from scratch five times with seeds {42, 123, 6512, 7, 69694} on the same split, this time with a 50-epoch budget. Table 2 reports the per-seed metrics, and Figure 4 visualizes the spread of MAEs and the per-run validation loss curves.

Table 2: Multi-run validation metrics. Five independent retrains of the tuned configuration on the same train/validation split, varying only the random seed. RMSE/atom is in kcal/mol/atom; all other metrics are in kcal/mol.

Seed	RMSE total	RMSE/atom	MAE total
42	5.711	0.777	4.088
123	5.284	0.740	3.699
6512	5.587	0.765	3.907
7	6.061	0.794	4.310
69694	5.911	0.810	4.171
Mean $\pm$ std	5.711 $\pm$ 0.300	0.777 $\pm$ 0.027	4.035 $\pm$ 0.238

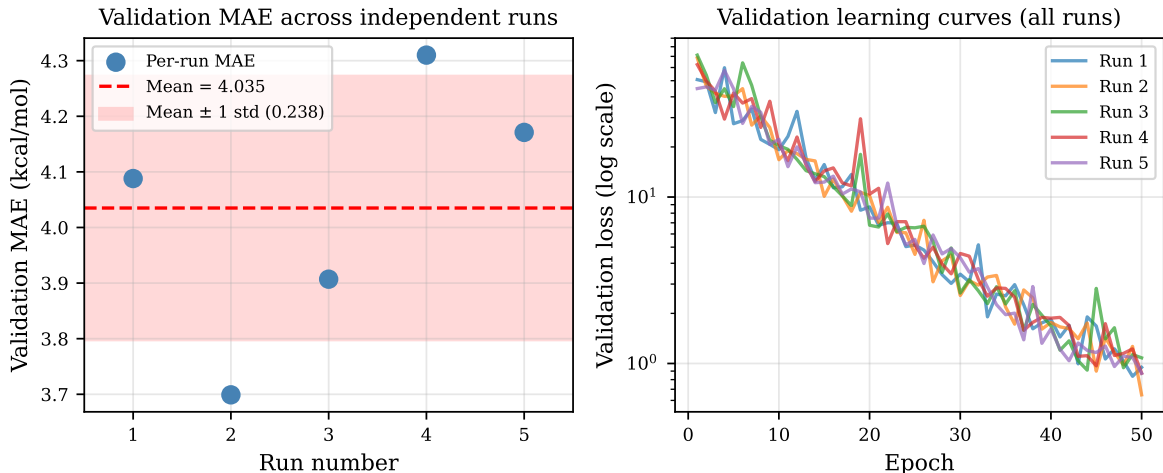


Figure 4: Multi-run stability. Left: per-run validation MAE across five independent retrainings. The dashed red line is the across-run mean and the shaded band is one standard deviation. Right: per-epoch validation loss curves for the same five runs on a log scale. The curves overlap closely throughout training, which indicates that the tuned configuration is stable under seed changes.

The standard deviation across seeds is small relative to the mean ($0.238/4.035 \approx 6\%$ for total MAE), and the validation learning curves overlap closely throughout training. This means the tuned configuration is genuinely stable under stochastic variation, and the final number is not a lucky initialization.

K-Fold Cross-Validation

The 3-fold cross-validation results are reported in Table 3 and visualized in Figure 5. Each fold was trained for 50 epochs from a fresh initialization, on the union of the other two folds, and evaluated on the held-out fold.

Table 3: Cross-validation metrics. 3-fold cross-validation with the tuned configuration. Each fold uses a fresh initialization and trains on the union of the other two folds.

Fold	RMSE total	RMSE/atom	MAE total
1	6.237	0.828	4.633
2	5.680	0.761	3.838
3	6.092	0.827	4.236
Mean $\pm$ std	6.003 ± 0.289	0.805 ± 0.039	4.236 ± 0.397

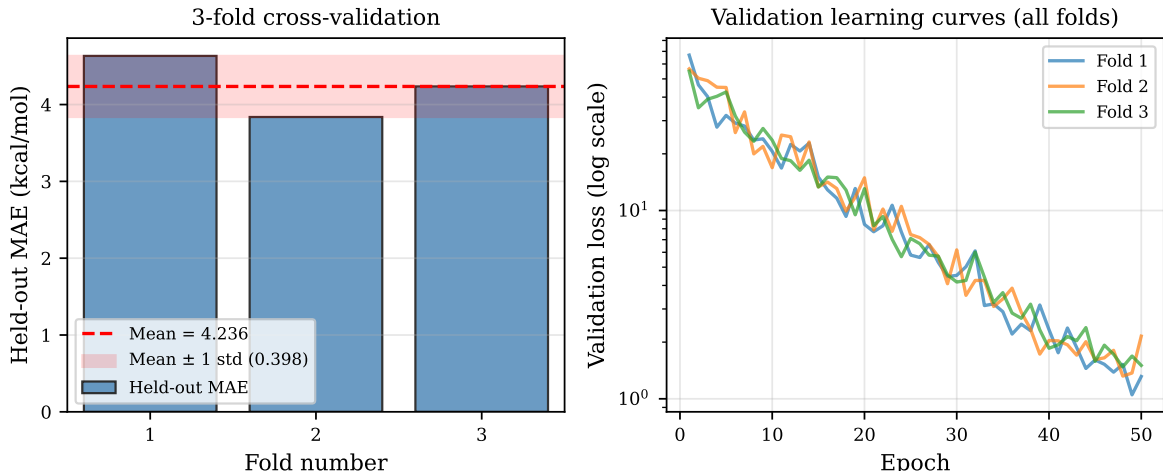


Figure 5: K -fold cross-validation. Left: per-fold held-out MAE for $K = 3$. The dashed red line is the across-fold mean, and the shaded band is one standard deviation. Right: per-epoch validation loss curves for each fold on a log scale. Curves are consistent across folds, which means the model is not exploiting any peculiarity of a single split.

The cross-validation mean MAE is slightly higher than the multi-run mean (4.236 vs 4.035 kcal/mol), and the standard deviation is also larger (0.397 vs 0.238). This is the expected pattern: the multi-run experiment captures only initialization and minibatch noise on a fixed split, whereas cross-validation also captures variance from the choice of held-out molecules. The per-fold validation curves in Figure 5 are visibly consistent, which confirms that no single fold is responsible for the spread.

Final Performance Summary and Comparison to ANI-1

Putting these results together, TwoStreamNet achieves a held-out MAE of 4.236 ± 0.397 kcal/mol with a corresponding total-energy RMSE of 6.003 ± 0.289 kcal/mol and a per-atom RMSE of 0.805 ± 0.039 kcal/mol/atom under 3-fold cross-validation. The multi-run experiment gives a tighter, single-split estimate of 4.035 ± 0.238 kcal/mol MAE and 5.711 ± 0.300 kcal/mol total-energy RMSE across 5 seeds.

For context, ANI-1 reports a total-energy RMSE of 1.2, 1.3, and 1.3 kcal/mol on its training, validation, and test splits respectively, and a 1.8 kcal/mol RMSE on the held-out GDB-10 extensibility set [3]. Comparing on the same metric, TwoStreamNet sits roughly 4 to 5 $\times$ above ANI-1.

Most of that gap is still attributable to training-set coverage. As noted in the methods, although the data-loading pipeline points at all eight shards, a file-extraction issue with s05 through s08 meant the actual training data fell back to the s01-s04 subset, that is, molecules with up to four heavy atoms only. ANI-1, by contrast, was trained on all of GDB s01 through s08, roughly 17.2 million conformations from $\sim 57k$ molecules with up to eight heavy atoms. Smith et al. show in Fig. 4E of the ANI-1 paper that the GDB-10 extensibility RMSE drops sharply each time a larger heavy-atom subset is added to training, going from 26.0 kcal/mol with up to four heavy atoms down to 1.8 kcal/mol once eight-heavy-atom molecules are included, with the inclusion of seven-heavy-atom mono-substituted aromatics alone accounting for the largest single drop [3]. In other words, the residual gap reflects a model that has only ever seen the smallest, least chemically

diverse end of the ANI-1 distribution, so a meaningful portion of the difference is attributable to data coverage rather than to the model itself.

Training-budget and loss-function choices contribute as well. ANI-1 trains until validation loss stops improving for 100 consecutive epochs and then repeats the optimization six times with an order-of-magnitude smaller learning rate each cycle, and it uses an exponential cost function the authors report yields 2 to 4 $\times$ lower error than a standard squared loss, whereas TwoStreamNet trains for at most 50 epochs per run under per-atom MSE.

With those caveats in mind, I would argue that TwoStreamNet still presents a reasonable, more modern reformulation of the core ANI idea. Whereas ANI-1 maintains a separate pyramidal MLP (768:128:128:64:1) per element with Gaussian activations and max-norm regularization, TwoStreamNet uses a single shared network that splits the AEV into its radial and angular halves, processes each through its own stream, and fuses the two via a squeeze-and-excitation gate before a residual stack. The regularization choices justified above (LayerNorm before every CELU, Xavier-uniform initialization, dropout, and L_2 weight decay) and the Bayesian-tuned hyperparameters give a principled starting point, and the production-mode estimate is grounded in 3-fold cross-validation rather than a single split.

Conclusion

Given the much smaller training domain and compute budget, I view a 6.003 ± 0.289 kcal/mol total-energy RMSE as a respectable starting point. The architectural changes (dual-stream AEV processing, squeeze-and-excitation gating, and a residual stack), the regularization stack (LayerNorm, dropout, L_2 weight decay, and early stopping), and the Bayesian tuning protocol together produce a stable, reproducible model whose generalization error is captured by both multi-run and cross-validation error bars. I expect that resolving the s05 through s08 loading issue, lengthening the training schedule, and adopting ANI-1’s exponential cost function would close most of the remaining gap to the ANI-1 reference numbers.

References

- [1] Friesner, R. A. (2005). *ab initio* Quantum Chemistry: Methodology and Applications. *Proceedings of the National Academy of Sciences*, 102(19), 6648-6653.
- [2] Behler, J., & Parrinello, M. (2007). Generalized Neural-Network Representation of High-Dimensional Potential-Energy Surfaces. *Physical Review Letters*, 98, 146401.
- [3] Smith, J. S., Isayev, O., & Roitberg, A. E. (2017). ANI-1: an extensible neural network potential with DFT accuracy at force field computational cost. *Chemical Science*, 8, 3192-3203.
- [4] Gao, X., Ramezanghorbani, F., Isayev, O., Smith, J. S., & Roitberg, A. E. (2020). TorchANI: A Free and Open Source PyTorch Based Deep Learning Implementation of the ANI Neural Network Potentials. *Journal of Chemical Information and Modeling*, 60(7), 3408-3415.
- [5] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, PMLR 9, 249-256.

- [6] Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv:1412.6980 [cs.LG]*.
- [7] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32*, 8024-8035.
- [8] Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-Excitation Networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 7132-7141.
- [9] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Identity Mappings in Deep Residual Networks. *European Conference on Computer Vision*, 630-645.
- [10] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2623-2631.